IA3 - Use-Case-to-Story Traceability Card

Student details

- Student name: Samuel Peter
- Student ID: 22410937
- GitHub username: Samuelpeterr
- Individual github repository: DCIT208-Portfolio/assignments/ia3-traceability-card/IA3_22410937_TraceabilityCard.pdf at main · Samuelpeterr/DCIT208-Portfolio
- Team name: nextgen_developers
- Project title: ABESDAC_Connect
- Client/project domain: Church Management System
- Date submitted:13/07/26

1. Selected use case

- Use case name: Donate and pay offering online
- Source evidence: requirement ID /REQ-5 "Members should be able to make donations and pay offertory online securely."
- team issue URL: Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-seg26-74-nextgen_dev: seg26-main-dcit208-client-project-2026-team-engineering-repository-seg26-dcit208-client-project-2026 created by GitHub Classroom
- Why this use case matters to the client: Why this use case matters to the client: This feature enables members to contribute financially to the church at any time, even if they are unable to attend services in person. It makes giving more convenient, increases accessibility, and helps the church receive donations and offerings securely

2. Use case card

- Actor: Church Member
- Goal: To make an online donation or pay an offering securely through the church website
- Trigger: The member clicks the Donate button.
- Preconditions:

-
    i. The church website is available.
2. The payment service is operational.
3. The member has a valid payment method.
4. The member has internet access.

Main success flow

1. The member opens the Donations/Offering page.
2. The member chooses whether to make a donation or pay an offering.
3. The member enters the amount and payment details.
4. The system validates the payment information.
5. The payment is processed successfully.
6. The system records the transaction.
7. The member receives a confirmation message and payment receipt.

Alternate flow

If the member wishes, they can choose a different payment method before confirming the transaction.

Exception / failure flow

If the payment is unsuccessful or payment details are invalid, the system displays an error message, does not record the transaction, and allows the member to try again.

Postconditions

1. The donation or offering payment is securely recorded.
2. The church's financial records are updated.
3. The member receives confirmation of the successful transaction.

3. User story

As a church member, I want to make online donations and pay my offerings securely, so that I can support the church from anywhere at anytime.

4. Acceptance criteria

AC1 - Successful path

- Successful path: Given the member is on the Donations and Offerings page with valid payment details, When the member submits the payment, Then the system processes the payment successfully, records the transaction, and displays a confirmation with a receipt.

AC2 - Error or exception path

Given the member submits invalid payment details or the payment cannot be processed, When the transaction is attempted, Then the system displays an appropriate error message, does not record the payment, and allows the member to try again.

5. Non-functional requirement link

- NFR category: Security
- NFR statement: The system shall encrypt all donation and offering payment data and use secure payment processing to protect users' financial and personal information.
- Why it matters for this use case: Members need confidence that their payment information is protected. Secure transactions help prevent unauthorized access, fraud, and data breaches.

6. GitHub issue link

- Team repository issue title: Implement Online Donations and Offering Payment
- Team repository issue URL: Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-seg26-74-nextgen_dev: seg26-main-dcit208-client-project-2026-team-engineering-repository-seg26-dcit208-client-project-2026 created by GitHub Classroom
- **7. Test idea**
- Test name: Online Donation Submission Test
- Input/context :Verify that a registered church member can successfully make an online donation or offering.
- Expected result: Expected Result: The system successfully processes the donation, stores the transaction details, and displays a confirmation message to the user.

8. Request-flow thinking

- Device/interface: Web browser (desktop or mobile)

- Route/screen/endpoint:: home to donate/offering page to payment page to confirmation page
- Data sent:1. Donation or offering amount

2. Payment method
3. Payment details
4. Transaction reference
5. Member ID (if logged in)

- Validation point:
-

    i. Amount must be greater than zero.

2. Required payment fields must be completed.
3. Payment details must be valid.

- Database table, collection or external service touched: Church database (stores transaction records)
- Response returned: The system confirms the payment, saves the transaction, and displays a receipt
- Failure point: If payment authorization fails or the internet connection is lost, the system displays an error message and does not save the transaction.
- Security/privacy concern: Payment information must be encrypted, sensitive payment details should not be stored unnecessarily, and only authorized users should have access to transaction records.

9. AI disclosure

- AI used? Yes/No: yes
- Tool used, if any : chatgpt
- What AI helped with:: Generating the traceability card, organizing the use case, user story, acceptance criteria, and improving the wording.
- What I changed:: Reviewed and edited the content to match the ABESDAC CONNECT project requirements and the team's design.
- What I personally verified :I verified that the use case, user story, acceptance criteria, and traceability links accurately represent the project requirements.

10. Integrity declaration

I understand the use case, user story, acceptance criteria, traceability links and request-flow reasoning submitted here, and I can explain them orally if called upon.

Typed name: Samuel Peter